

EPWM Configuration

Overview

This codebase is designed to configure and control the Enhanced Pulse Width Modulation (ePWM) modules on a Texas Instruments microcontroller (likely from the C2000 series, given the use of ``driverlib.h`` and ``device.h``). The code initializes ePWM modules (ePWM1, ePWM2, and ePWM3), configures their signal parameters (frequency, duty cycle, and phase), and sets up GPIO pins for ePWM output. The code is structured across multiple files: ``EPWM_config.h``, ``duty.h``, ``duty.c``, and ``main.c``.

The primary purpose is to generate synchronized PWM signals with specific phase shifts (e.g., for motor control or power electronics applications). The code uses the Texas Instruments DriverLib library to interface with the microcontroller's peripherals.

File Descriptions and Code

1. ``EPWM_config.h``

This header file defines constants, macros, and function prototypes for configuring ePWM modules and their associated GPIO pins.

Key Components:

Macros:

Defines base addresses for ePWM modules: ``myEPWM1_BASE``, ``myEPWM2_BASE``, and ``myEPWM3_BASE``.

Specifies GPIO pin configurations for ePWM1A and ePWM1B (pins 0 and 1, respectively).

Commented-out sections include configurations for additional ePWM modules (ePWM2, ePWM3, ePWM4) and a duty cycle buffer (``dutyBuffer``) for dynamic updates.

Function Prototypes:

``EPWM_init()``: Initializes the ePWM modules (though not implemented in the provided code).

``configureEPWMs()``: Configures the ePWM modules for signal generation.

``configurePhase()``: Sets the phase shift for a given ePWM module relative to a master module.

``updateDutyCycle()``: Updates the duty cycle of an ePWM module (commented out).

Code:

```
#ifndef EPWM_CONFIG_H
#define EPWM_CONFIG_H

#include "driverlib.h"

#include "device.h"

// #include "board.h"
```

```

#define myEPWM1_BASE EPWM1_BASE
#define myEPWM2_BASE EPWM2_BASE
#define myEPWM3_BASE EPWM3_BASE
#define GPIO_PIN_EPWM1_A 0
#define GPIO_PIN_CFG_EPWM1_A GPIO_0_GPIO0
#define GPIO_PIN_EPWM1_B 1
#define GPIO_PIN_CFG_EPWM1_B GPIO_1_GPIO1

// Function prototypes
void EPWM_init(void);
void configureEPWMs(void);
void configurePhase(uint32_t base, uint32_t masterBase, uint16_t phaseVal);
#endif // EPWM_CONFIG_H

```

Notes:

The commented-out sections suggest that the code is a work-in-progress or tailored for a specific use case (e.g., only ePWM1 is fully configured).

The GPIO pin definitions ensure that the ePWM outputs are mapped to specific microcontroller pins.

2. `duty.h`

This header file defines the global PWM signal parameters and includes the implementation of the `EPWM1\_init()` and `configureEPWMs()` functions.

Key Components:

Global Structure:

`pwmSignal`: An `EPWM\_SignalParams` structure that defines the PWM signal characteristics:

Period: 10,000 cycles (sets PWM frequency).

Duty Cycle: 50% for both A and B channels (`0.50f`).

Deadband Enabled: `true` (indicates deadband control is active for complementary outputs).

System Clock Frequency: `DEVICE\_SYSCCLK\_FREQ` (device-specific clock frequency).

Counter Mode: Up-down counting mode (`EPWM\_COUNTER\_MODE\_UP\_DOWN`).

Clock Dividers: No prescaling (`EPWM\_CLOCK\_DIVIDER\_1`, `EPWM\_HSCLOCK\_DIVIDER\_1`).

Function: `EPWM1\_init():`

Configures GPIO pins for ePWM1A and ePWM1B (both mapped to GPIO0 in the provided code, which may be a typo or intentional for a specific setup).

Sets the pin type to standard (`GPIO\_PIN\_TYPE\_STD`) and synchronization mode (`GPIO\_QUAL\_SYNC`).

Function: `configureEPWMs()`:

Configures ePWM1 using the `pwmSignal` parameters via `EPWM\_configureSignal()`.

Commented-out lines indicate potential configuration for ePWM2 and ePWM3.

Sets up ePWM1 as the master module with synchronization output on counter zero (`EPWM\_SYNC\_OUT\_PULSE\_ON\_CNTR\_ZERO`).

Configures phase shifts for ePWM2 (120°) and ePWM3 (240°) relative to ePWM1 using `configurePhase()`.

Enables phase shift loading for ePWM2 and ePWM3 and sets their sync input to ePWM1's sync output.

Function: `configurePhase()`:

Calculates the phase shift value based on the master module's period and desired phase angle (in degrees).

Adjusts the phase register value depending on the counter mode (up-down or otherwise).

Sets the phase shift and initial counter value for the specified ePWM module.

Code:

```
#include "duty.h"

#include "driverlib.h"

#include "device.h"

// #include "board.h"

// Global PWM configuration parameters
EPWM_SignalParams pwmSignal =
    {10000, 0.50f, 0.50f, true, DEVICE_SYSCLK_FREQ,
     EPWM_COUNTER_MODE_UP_DOWN, EPWM_CLOCK_DIVIDER_1,
     EPWM_HSCLOCK_DIVIDER_1};

// extra part for CCS
void EPWM1_init(){
    GPIO_setPinConfig(GPIO_PIN_CFG_EPWM1_A); // ePWM1A mapped to GPIO0
    GPIO_setPadConfig(0, GPIO_PIN_TYPE_STD);
    GPIO_setQualificationMode(0, GPIO_QUAL_SYNC);
    GPIO_setPinConfig(GPIO_PIN_CFG_EPWM1_B); // ePWM1A mapped to GPIO0
    GPIO_setPadConfig(0, GPIO_PIN_TYPE_STD);
```

```

    GPIO_setQualificationMode(0, GPIO_QUAL_SYNC);
}

// configureEPWMs - Configures EPWM1, EPWM2, EPWM3
void configureEPWMs(void)
{
    // Configure EPWM1, EPWM2, EPWM3 for desired frequency & duty
    EPWM_configureSignal(myEPWM1_BASE, &pwmSignal);
    // EPWM_configureSignal(myEPWM2_BASE, &pwmSignal);
    // EPWM_configureSignal(myEPWM3_BASE, &pwmSignal);
    //EPWM_configureSignal(myEPWM4_BASE, &pwmSignal);
    // Configure phase relationships
    EPWM_disablePhaseShiftLoad(myEPWM1_BASE);
    EPWM_setPhaseShift(myEPWM1_BASE, 0U);
    // ePWM1 SYNCO generated on CTR=0
    EPWM_enableSyncOutPulseSource(myEPWM1_BASE,
                                   EPWM_SYNC_OUT_PULSE_ON_CNTR_ZERO);
    // Configure phase shift for EPWM2 & EPWM3 (120°, 240°)
    configurePhase(myEPWM2_BASE, myEPWM1_BASE, 120);
    configurePhase(myEPWM3_BASE, myEPWM1_BASE, 240);

    EPWM_setSyncInPulseSource(myEPWM2_BASE,
    EPWM_SYNC_IN_PULSE_SRC_SYNCOUT_EPWM1);

    EPWM_setSyncInPulseSource(myEPWM3_BASE,
    EPWM_SYNC_IN_PULSE_SRC_SYNCOUT_EPWM1);

    EPWM_enablePhaseShiftLoad(myEPWM2_BASE);

    EPWM_enablePhaseShiftLoad(myEPWM3_BASE);
}

//
// configurePhase - Configure ePWM phase shift
//
void configurePhase(uint32_t base, uint32_t masterBase, uint16_t phaseVal)
{
    uint32_t readPrdVal, phaseRegVal;

```

```

// Read Period value to calculate phase register value
readPrdVal = EPWM_getTimeBasePeriod(masterBase);

// Calculate phase value based on counter mode
if((HWREGH(base + EPWM_O_TBCTL) & 0x3U) == EPWM_COUNTER_MODE_UP_DOWN)
{
    phaseRegVal = (2U * readPrdVal * phaseVal) / 360U;
}
else
{
    phaseRegVal = (readPrdVal * phaseVal) / 360U;
}

EPWM_selectPeriodLoadEvent(base, EPWM_SHADOW_LOAD_MODE_SYNC);

EPWM_setPhaseShift(base, phaseRegVal);

EPWM_setTimeBaseCounter(base, phaseRegVal);
}

```

Notes:

The phase shift configuration suggests a three-phase application (e.g., motor control), with ePWM2 and ePWM3 offset by 120° and 240°, respectively.

The `EPWM1\_init()` function contains a potential error: both ePWM1A and ePWM1B are configured to GPIO0, which may require correction.

3. `duty.c` and `main.c`

These files contain the main program logic for initializing the microcontroller and running the ePWM configuration.

Key Components:

Main Function:

Initializes the device clock and peripherals (`Device\_init()`).

Disables GPIO pin locks and enables internal pull-ups (`Device\_initGPIO()`).

Initializes the Peripheral Interrupt Expansion (PIE) module and vector table (`Interrupt\_initModule()`, `Interrupt\_initVectorTable()`).

Calls `Board\_init()` to initialize GPIO pins for ePWM modules (implementation not shown but assumed to be in `board.h`).

Disables the ePWM clock synchronization
 (`SysCtl\_disablePeripheral(SYSCTL\_PERIPH\_CLK\_TBCLKSYNC)`).

Calls `configureEPWMs()` to set up the ePWM modules.

Re-enables ePWM clock synchronization
(`SysCtl_enablePeripheral(SYSCTL_PERIPH_CLK_TBCLKSYNC)`).

Enables the ePWM1 interrupt (`Interrupt_enable(INT_EPWM1)`) and global interrupts (`EINT`, `ERTM`).

Enters an infinite idle loop with a no-operation instruction (`NOP`).

Commented-out code suggests periodic duty cycle updates for ePWM1 (e.g., switching between 25% and 75% duty cycles every second).

Code:

```
#include "driverlib.h"

#include "device.h"

#include "board.h"

#include "duty.h"

void main(void)
{
    // Initialize device clock and peripherals
    Device_init();

    // Disable pin locks and enable internal pull-ups
    Device_initGPIO();

    // Initialize PIE and clear PIE registers. Disables CPU interrupts
    Interrupt_initModule();

    // Initialize the PIE vector table
    Interrupt_initVectorTable();

    // Init GPIO pins for ePWM1, ePWM2, ePWM3
    Board_init();

    // Disable sync(Freeze clock to PWM as well)
    SysCtl_disablePeripheral(SYSCTL_PERIPH_CLK_TBCLKSYNC);

    // Configure PWM signals
    configureEPWMs();

    // Enable sync and clock to PWM
    SysCtl_enablePeripheral(SYSCTL_PERIPH_CLK_TBCLKSYNC);

    // Enable interrupts required for this example
```

```

Interrupt_enable(INT_EPWM1);

// Enable global interrupts

EINT;

ERTM;

// Idle loop

for(;;)

{

    asm (" NOP");

    // DEVICE_DELAY_US(1000000); // 1 second

}

}

```

Notes:

The commented-out duty cycle updates indicate that dynamic PWM control was planned but not implemented.

The interrupt for ePWM1 is enabled, but no interrupt service routine (ISR) is provided in the code, suggesting additional code may exist elsewhere.

Working of the Code

Initialization

1. Device Setup:

The ``main()`` function initializes the microcontroller's clock, peripherals, GPIO, and interrupt system.

The ePWM clock is temporarily disabled to prevent unintended behavior during configuration.

2. GPIO Configuration:

The ``EPWM1_init()`` function maps ePWM1A and ePWM1B to GPIO pins (though both are currently set to GPIO0, which may need correction).

``Board_init()`` (not shown) likely configures additional GPIO pins for ePWM2 and ePWM3.

3. ePWM Configuration:

The ``configureEPWMs()`` function sets up ePWM1 as the master module with a 10,000-cycle period and 50% duty cycle.

ePWM2 and ePWM3 are configured to follow ePWM1 with phase shifts of 120° and 240°, respectively, ensuring synchronized three-phase PWM signals.

The up-down counter mode is used, which is common in applications requiring symmetrical PWM signals (e.g., for motor control or inverters).

4. Phase Synchronization:

The `configurePhase()` function calculates the phase shift value based on the master module's period and the desired phase angle.

The phase shift is applied to ePWM2 and ePWM3, ensuring proper timing for three-phase applications.

5. Runtime Behavior:

After configuration, the ePWM modules generate continuous PWM signals.

The main loop is idle (`NOP`), but commented-out code suggests the possibility of dynamically updating the duty cycle.

The enabled ePWM1 interrupt implies that an ISR (not shown) may handle events such as counter overflows or specific PWM conditions.

Intended Application

The code is likely intended for a three-phase power electronics or motor control application, where ePWM1, ePWM2, and ePWM3 generate synchronized PWM signals with 120° phase offsets. The use of up-down counting and deadband control suggests compatibility with applications requiring complementary PWM outputs (e.g., for driving power transistors in an inverter).

Potential Issues and Improvements

1. GPIO Configuration Error:

In `EPWM1_init()`, both ePWM1A and ePWM1B are mapped to GPIO0, which is likely incorrect. These should be mapped to distinct pins (e.g., GPIO0 and GPIO1, as defined in `EPWM_config.h`).

Suggested Fix: Update the second `GPIO_setPinConfig()` call to use `GPIO_PIN_CFG_EPWM1_B` and correct the `GPIO_setPadConfig()` and `GPIO_setQualificationMode()` calls to use GPIO1.

2. Commented-Out Code:

The commented-out configurations for ePWM2, ePWM3, and ePWM4, as well as the duty cycle update logic, suggest incomplete implementation or selective enabling. These should be reviewed and either implemented or removed.

3. Interrupt Handling:

The ePWM1 interrupt is enabled, but no ISR is provided. Ensure that an appropriate ISR is implemented to handle ePWM1 events.

4. Scalability:

The code is specific to ePWM1, ePWM2, and ePWM3. For scalability, consider parameterizing the configuration to support additional ePWM modules dynamically.

5. Documentation:

Add comments to clarify the purpose of each function and macro.

Document the expected system clock frequency (`DEVICE_SYSCLK_FREQ`) and its impact on PWM frequency.

Example Usage

To use this code:

1. Ensure the Texas Instruments DriverLib and device-specific headers (`driverlib.h`, `device.h`, `board.h`) are included in the project.
2. Correct the GPIO configuration in `EPWM1_init()` if necessary.
3. Implement the `Board_init()` function or include the appropriate board support file.
4. Add an ISR for the ePWM1 interrupt if required.
5. If dynamic duty cycle updates are needed, uncomment and refine the `updateDutyCycle()` calls in the main loop.
6. Compile and flash the code onto a compatible Texas Instruments microcontroller.

Conclusion

This codebase provides a foundation for generating synchronized three-phase PWM signals using the ePWM modules on a Texas Instruments microcontroller. With minor corrections and potential enhancements, it can be used for applications such as motor control, power inverters, or other PWM-based systems. The modular structure (header files, configuration functions, and main loop) allows for easy extension to additional ePWM modules or advanced features like dynamic duty cycle control.

Detailed Line-by-Line Explanation

Header Files

```
#include "duty.h"
```

```
#include "driverlib.h"
```

```
#include "device.h"
```

```
//#include "board.h"
```

- **duty.h**: This is your custom header file. It might contain macro definitions (like `myEPWM1_BASE`, constants, or helper functions).
- **driverlib.h**: TI's DriverLib API. This provides ready-to-use functions like `GPIO_setPinConfig()`, `EPWM_configureSignal()`, etc. Instead of writing raw register code, DriverLib makes things easier and less error-prone.
- **device.h**: Configures system-level things like clock frequency, watchdog timer, enabling/disabling peripherals.
- **board.h (commented out)**: Usually contains pin multiplexing and board-level initialization (depends on your project).

Global PWM Configuration

```
EPWM_SignalParams pwmSignal =
```

```
    {10000, 0.50f, 0.50f, true, DEVICE_SYSCLK_FREQ,  
     EPWM_COUNTER_MODE_UP_DOWN, EPWM_CLOCK_DIVIDER_1,  
     EPWM_HSCLOCK_DIVIDER_1};
```

This structure (`EPWM_SignalParams`) defines **how the PWM should look**. Each field has a meaning:

1. **10000** → Desired PWM frequency (10 kHz).
 - Common choice for motor control and inverters.
 - High enough for smooth control, low enough to avoid excessive switching losses.
2. **0.50f** → Duty cycle = 50%.
 - Means PWM output is ON for half the period.
3. **0.50f** → Deadband/phase duty (used in complementary PWM).
4. **true** → Enable PWM output.
5. **DEVICE_SYSCLK_FREQ** → System clock frequency.
 - DriverLib will compute timer period from this.

6. **EPWM_COUNTER_MODE_UP_DOWN** → Up-down counting mode.
 - Produces **center-aligned PWM** (symmetric waveform).
 - Reduces harmonics compared to edge-aligned PWM.
 7. **EPWM_CLOCK_DIVIDER_1** and **EPWM_HSCLOCK_DIVIDER_1** → No clock division.
 - PWM uses full system clock speed.
-

Pin Initialization

```
void EPWM1_init(){
```

```
    GPIO_setPinConfig(GPIO_PIN_CFG_EPWM1_A); // ePWM1A mapped to GPIO0
```

```
    GPIO_setPadConfig(0, GPIO_PIN_TYPE_STD);
```

```
    GPIO_setQualificationMode(0, GPIO_QUAL_SYNC);
```

```
    GPIO_setPinConfig(GPIO_PIN_CFG_EPWM1_B); // ePWM1B mapped to GPIO1
```

```
    GPIO_setPadConfig(0, GPIO_PIN_TYPE_STD);
```

```
    GPIO_setQualificationMode(0, GPIO_QUAL_SYNC);
```

```
}
```

- **GPIO_setPinConfig()**: Maps the PWM function to a physical pin.
 - EPWM1A → GPIO0
 - EPWM1B → GPIO1
 - **GPIO_setPadConfig(0, GPIO_PIN_TYPE_STD)**: Configures GPIO0 as a standard push-pull digital output.
 - **GPIO_setQualificationMode(0, GPIO_QUAL_SYNC)**: Synchronizes input/output signals to the system clock (helps avoid glitches).
-

Main ePWM Configuration

```
void configureEPWMs(void)
```

```
{
```

```
    EPWM_configureSignal(myEPWM1_BASE, &pwmSignal);
```

- Configures **EPWM1** with the parameters in pwmSignal.
- This sets frequency = 10 kHz, duty = 50%, center-aligned.
- could be used to configure **EPWM2**, **EPWM3**, **EPWM4** similarly.

- Here, only EPWM1 is explicitly configured, while EPWM2/3 will be adjusted later for phase shift.

```
EPWM_disablePhaseShiftLoad(myEPWM1_BASE);
```

```
EPWM_setPhaseShift(myEPWM1_BASE, 0U);
```

- **EPWM_disablePhaseShiftLoad()**: Makes EPWM1 the **master PWM**.
- **EPWM_setPhaseShift(0)**: No delay, EPWM1 starts at time = 0.

```
EPWM_enableSyncOutPulseSource(myEPWM1_BASE,  
                               EPWM_SYNC_OUT_PULSE_ON_CNTR_ZERO);
```

- EPWM1 generates a **sync pulse** whenever its counter resets (CTR = 0).
- This sync pulse can be shared with other ePWM modules.

```
configurePhase(myEPWM2_BASE, myEPWM1_BASE, 120);
```

```
configurePhase(myEPWM3_BASE, myEPWM1_BASE, 240);
```

- Configures **EPWM2** to be shifted by 120°.
- Configures **EPWM3** to be shifted by 240°.

```
EPWM_setSyncInPulseSource(myEPWM2_BASE,  
EPWM_SYNC_IN_PULSE_SRC_SYNCOUT_EPWM1);
```

```
EPWM_setSyncInPulseSource(myEPWM3_BASE,  
EPWM_SYNC_IN_PULSE_SRC_SYNCOUT_EPWM1);
```

- Both EPWM2 and EPWM3 use **EPWM1's sync-out pulse** as their trigger.
- This way, they remain phase-locked to EPWM1.

```
EPWM_enablePhaseShiftLoad(myEPWM2_BASE);
```

```
EPWM_enablePhaseShiftLoad(myEPWM3_BASE);
```

```
}
```

- Applies the calculated phase shifts for EPWM2 and EPWM3.
 - Without this, the values wouldn't take effect.
-

Phase Calculation

```
void configurePhase(uint32_t base, uint32_t masterBase, uint16_t phaseVal)
```

```
{
```

```
    uint32_t readPrdVal, phaseRegVal;
```

```
    readPrdVal = EPWM_getTimeBasePeriod(masterBase);
```

- Reads the **PWM period (in counts)** from the master (EPWM1).
- Example: If system clock = 100 MHz, frequency = 10 kHz → period = 5000 counts.

```
    if((HWREGH(base + EPWM_O_TBCTL) & 0x3U) == EPWM_COUNTER_MODE_UP_DOWN)
```

```
    {
```

```
        phaseRegVal = (2U * readPrdVal * phaseVal) / 360U;
```

```
    }
```

```
    else
```

```
    {
```

```
        phaseRegVal = (readPrdVal * phaseVal) / 360U;
```

```
    }
```

- Converts **phase in degrees** → **counter offset**.
- For **up-down mode**: multiply by 2 (since counter counts up and down).
- Example: Period = 5000 counts
 - 120° shift → $(2 * 5000 * 120) / 360 = 3333$ counts
 - 240° shift → $(2 * 5000 * 240) / 360 = 6666$ counts

```
    EPWM_selectPeriodLoadEvent(base, EPWM_SHADOW_LOAD_MODE_SYNC);
```

```
    EPWM_setPhaseShift(base, phaseRegVal);
```

```
    EPWM_setTimeBaseCounter(base, phaseRegVal);
```

```
}
```

- **EPWM_selectPeriodLoadEvent()**: Phase shift updates only during sync events (avoids glitches).
 - **EPWM_setPhaseShift()**: Loads calculated offset into the PWM's phase register.
 - **EPWM_setTimeBaseCounter()**: Aligns the counter to start at the right point.
-

Final Understanding

- **EPWM1**: Master PWM (0° phase, sync generator).
- **EPWM2**: Slave PWM, shifted by 120°.
- **EPWM3**: Slave PWM, shifted by 240°.
- All signals = 10 kHz, 50% duty, center-aligned.